

융합보안 프리캡스톤 디자인

AI 기반 문서 유사도 판별 시스템



목차

table of contents

01
중간발표
피드백

02
회의 진행

03
팀장님
피드백

04
전체
파이프라인

05
진행 상황

06
진행 계획



조별 피드백

벡터 스토리지 사용 추천

문서끼리만 비교해도 된다

BGE-M3-K 추천

피드백 이해

- 기존에는 임베딩된 npy 파일과 기존 문서 json 파일을 서버에 저장하여 비교하는 방식을 사용
- 시간이 너무 많이 소요
- json 파일 없이 임베딩된 결과로만 비교하면 되고, 이 임베딩된 파일을 벡터 스토리지에 저장하면 시간 절약 가능

피드백

중간 발표 당시 피드백

조별 피드백

벡터 스토리지 사용 추천

문서끼리만 비교해도 된다

BGE-M3-K 추천

피드백 이해

- 기존에는 문서 내부의 문장 한줄한줄 비교하여 두 문서 사이의 유사도가 몇 퍼센트인지를 중점으로 진행
- 피드백으로, 1개의 외부 문서와 30개의 내부 문서를 전체적으로 비교해, 30개의 문서 중 **제일 유사한 문서를 뽑아내기**만 해도 된다는 피드백
- 특히 어느 문장이 유사한지를 판별하지 않아도 돼서 **시간 절약 가능**



조별 피드백

벡터 스토리지 사용 추천

문서끼리만 비교해도 된다

BGE-M3-K 추천

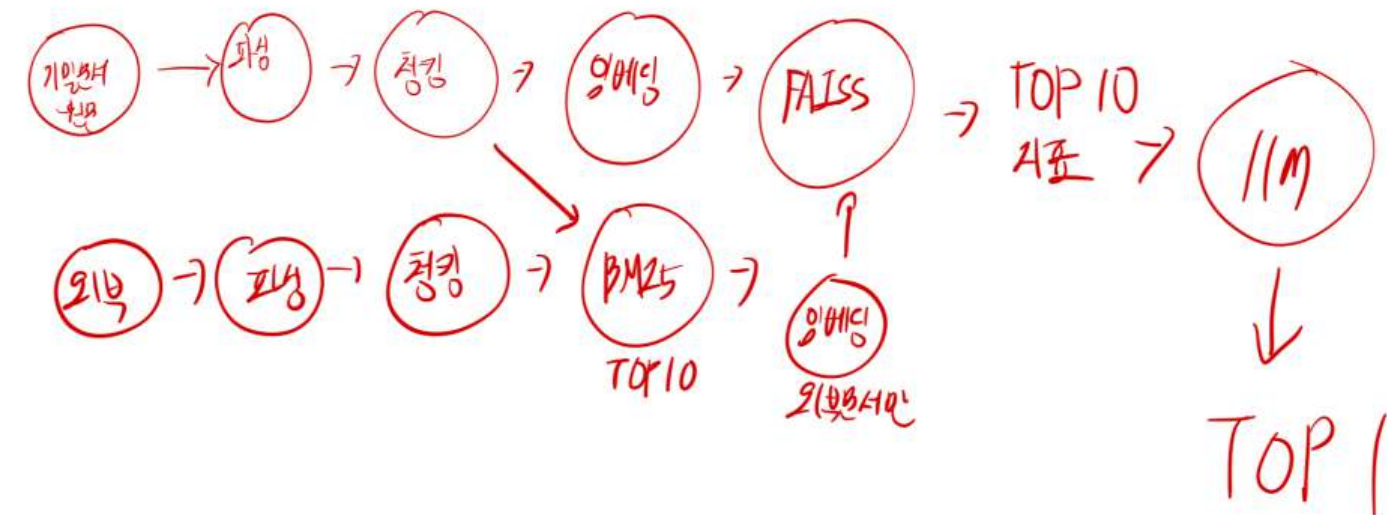
피드백 이해

- 기존에는 임베딩 모델을 BGE-M3를 사용
- **한글 버전에 최적화된 BGE-M3-K**를 사용해보라는 피드백
- 임베딩 모델 말고도 벡터 스토어 등등 다른 것에 대한 모델도 여러 개 사용해보라는 피드백



회의 안건 1

기존에 진행하던 방향과 목표가 달라졌기 때문에
전체적으로 파이프라인을 재구성하고 방향 설정을 진행



회의를 통해 재구성한 파이프라인



회의 안건 2

다양한 모델을 사용해보고 시행착오를 겪어보라는
팀장님의 피드백에 맞춰 시도해볼 모델 선정

파싱

- 일단 한줄로

청킹

- 문장 단위, 토큰 단위, 오버랩 윈도우 기반 0

파싱, 청킹 분리

임베딩

- BGE-M3 (1024)
- BGE-M3-K (1024)
- KoSimCSE (768, 한국어 특화)
- E5-KR (768, 한국어 특화)

임베딩 모델



회의 안건 2

다양한 모델을 사용해보고 시행착오를 겪어보라는
팀장님의 피드백에 맞춰 시도해볼 모델 선정

- FAISS (밀집)
- Chroma -한
- 쿼더런트 -유 (희소, 밀집 둘다 가능)
- Pinecone (희소, 밀집 둘다 가능)
- Milvus

벡터스토어 모델 선정



회의 안건 3

중간발표 회식 때 추천해주신 **BM25**에 대한 조사

✿ BM25 점수 공식

$$\text{score}(q, D) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot (1 - b + b \cdot \frac{|D|}{\text{avg}D})}$$

- $f(t, D)$: 단어 t 의 문서 내 빈도
- $|D|$: 문서 길이
- $\text{avg}D$: 평균 문서 길이
- k_1 : TF 가중 조정 (보통 1.2~2.0)
- b : 문서 길이 보정 비율 (보통 0.75)

BM25 공식

- 정보 검색(IR) 분야에서 가장 널리 쓰이는 **문서 유사도 계산 알고리즘**
- 단순한 단어 빈도(TF-IDF)를 개선하여 문서 길이와 단어 중요도를 함께 고려

BM25란?



구성한 파이프라인에 대한 피드백을 받기 위해 메일 전송

보낸 내용

1. 코드를 작성하기 전에 새로운 구성안이
적절한 방향인지 확인 받기

2. BM25의 사용 여부

피드백

[사전 기밀 문서 임베딩]

- 1) 문서내 TEXT 추출
- 2) 문서 청킹
- 3) 문서 임베딩 벡터 생성
- 4) BM25 키워드 기반 벡터 생성
- 5) 용어집 생성 및 용어집 업데이트
- 6) Milvus, Pinecone, Qdrant 등 다양한 임베딩 벡터 저장소에 3), 4)의 결과를 저장



구성한 파이프라인에 대한 피드백을 받기 위해 메일 전송

보낸 내용

1. 코드를 작성하기 전에 새로운 구성안이
적절한 방향인지 확인 받기

2. BM25의 사용 여부

피드백

[문서 유출 시도]

1) 문서내 TEXT 추출

2) 문서 청킹

3) 문서 임베딩 벡터 생성

4) BM25 키워드 기반 벡터 생성

5) 3) 으로 Top N 개 추출 4) 로 Top N 개 추출

6) 5) 의 결과로 Hybrid하여 점수 계산 (키워드 비중 및 유사도 비중 고려)

7) 특정 점수 기준 이상이면 유사 문서로 판단 하여 결과 출력



구성한 파이프라인에 대한 피드백을 받기 위해 메일 전송

보낸 내용

1. 코드를 작성하기 전에 새로운 구성안이 적절한 방향인지 확인 받기

-
2. BM25의 사용 여부

피드백

[결과 문서 LLM 답변 생성] - 단계 생략 가능

1) 7)의 결과 목록을 LLM에게 전달하여 최종 사용자에게 전달할 답변 생성

==> [결과 문서 LLM 답변 생성] 부분은 Optional입니다.

==> [문서 유출 시도]의 7)의 결과로 적절한 결과 화면을 생성하면 될 것 같습니다.



구성한 파이프라인에 대한 피드백을 받기 위해 메일 전송

보낸 내용

1. 코드를 작성하기 전에 새로운 구성안이 적절한 방향인지 확인 받기

2. BM25의 사용 여부

피드백

BM25 스코어는 문서에 포함된 단어들(term)에 대해서만 계산되므로, 전체 어휘사전(vocabulary) 크기의 벡터에서 대부분의 값이 0이 됩니다. 따라서 Sparse Vector(희소 벡터)라고 명명되며, **sparse 벡터 검색이 가능한 벡터 스토리지를 이용**해야 합니다. 아쉽게도 Faiss는 희소(sparse) 벡터 검색을 기본적으로 지원하지 않습니다. Faiss는 밀집(dense) 벡터의 효율적인 유사도 검색에 특화된 라이브러리입니다.



구성한 파이프라인에 대한 피드백을 받기 위해 메일 전송

보낸 내용

1. 코드를 작성하기 전에 새로운 구성안이 적절한 방향인지 확인 받기

2. BM25의 사용 여부

피드백

[문서 유출 판단 프로그램 실행]

1) 용어집 로드 (메모리에 1회 로드 하여 성능 이슈가 없도록 처리)

[참고사항]

- BM25 용어집을 별도 관리해야 하며, **indices, values** 쌍을 직접 생성하여 **벡터 스토리지에 저장**해 주어야 합니다.
- 사전에 **sparse** 벡터를 미리 저장해 놔야 유출 시도 문서의 키워드를 즉시 검색할 수 있습니다.
- 사전 기밀 문서 임베딩에 새로운 문서가 추가될 경우 [문서 유출 판단 프로그램 실행]의 용어집을 새로 로드 해야 합니다.



구성한 파이프라인에 대한 피드백을 받기 위해 메일 전송

보낸 내용

1. 코드를 작성하기 전에 새로운 구성안이 적절한 방향인지 확인 받기

2. BM25의 사용 여부

피드백

BM25를 사용하지 않는다면 코사인 유사도만으로 문서 유사도를 판단해야 하는데, 동일 도메인의 유사한 문서들끼리의 코사인 유사도 추출 결과가 얼마나 좋을지 의문입니다.

대부분의 문서들에 유사한 내용들이 포함될 확률이 높기 때문에 변별력이 떨어질 수 있을 것 같습니다.

BM25에 대한 설명

개념을 쉽게 이해하기 위해, 전체 어휘사전(Vocabulary) 크기가 15개로 매우 작다고 가정해 보겠습니다.

1. 전체 어휘사전 (Vocabulary) 각 단어(term)는 고유한 숫자 ID(index)를 가집니다.

- "the": 0
- "quick": 1
- "brown": 2
- "fox": 3
- "jumps": 4
- "over": 5
- "lazy": 6
- "dog": 7
- "apple": 8
- "banana": 9
- "computer": 10
- "science": 11
- "is": 12
- "fun": 13
- "data": 14

2. 분석할 문서 (Document) 이제 BM25 스코어를 계산할 문서는 다음과 같습니다. doc = "A quick brown fox and a lazy dog."

3. BM25 스코어 계산 (가상) 이 문서에 대해 BM25 스코어를 계산하면, 문서에 실제로 존재하는 단어들에 대해서만 0이 아닌 스코어가 나옵니다. (여기서 스코어는 예시를 위한 가상의 값입니다.)

- "quick" (index 1): 2.3
- "brown" (index 2): 2.5
- "fox" (index 3): 3.1
- "lazy" (index 6): 2.8
- "dog" (index 7): 2.7
- ... 나머지 "the", "apple", "banana", "computer" 등 (indices 0, 4, 5, 8, 9, 10, 11, 12, 13, 14)의 스코어는 모두 0.0 입니다.

indices 와 values 생성

위의 결과에서 스코어가 0이 아닌 것들만 뽑아서 indices와 values 리스트를 만듭니다.

1. indices (인덱스 리스트) 스코어가 0이 아닌 단어들의 고유 ID(index)를 모은 리스트입니다.

- indices = [1, 2, 3, 6, 7]

2. values (스코어 리스트) indices 리스트와 동일한 순서로 해당 단어들의 실제 BM25 스코어를 모은 리스트입니다.

- values = [2.3, 2.5, 3.1, 2.8, 2.7]



피드백의 피드백을 받기 위해 메일 전송

보낸 내용



1. 기존 DLP와의 차별성?

2. LLM의 사용



피드백

제가 알고 있는 DLP는 Data Loss Prevention 으로 내부 정보 유출 방지 개념으로 알고 있으며, 정보 유출 방법에는 여러가지가 있고, 그 중에서 문서 유사도 방식으로 유출 방지 하는 것으로 생각하면 될 것 같습니다. DLP와의 차별성을 비교하기 보다는 subset 느낌이 아닐까 싶습니다.



피드백의 피드백을 받기 위해 메일 전송

보낸 내용

1. 기존 DLP와의 차별성?

2. LLM의 사용

피드백

=> 문서 유사도 비교 판단이 핵심 주제이며 LLM은 양념이나 데코레이션 용도로 사용하면 될 것 같습니다.

=> 일반적인 RAG 워크 플로우를 생각해보면 여기서도 핵심은 사용자 질문에 대해 정확한 문서를 찾는 부분이 핵심이며, 질문 의도에 제일 적합한 답변을 뽑은 후 LLM에게 니가 잘 답변해봐라 의 형태입니다. LLM은 질문과 답변을 모두 받아서 그럴싸하고 정리된 답변을 생성하는 용도로만 사용됩니다.

파이프라인

최종 로직



여러 피드백을 수용해 최종 발표때까지 구현할 프로젝트 파이프라인

기밀 문서 (사전에 구현)	외부 문서 (업로드 시)	유출 시도 시 로직
문서 파서	문서 파서	(1)임베딩 파일끼리 코사인 유사도 판별
문서 청킹	문서 청킹	(2)BM25를 이용해 단어 유사도 판별
청킹된 json 임베딩	청킹된 json 임베딩	(1)과 (2)에 대한 Hybrid 유사도
BM25에 대한 용어집에 로드	BM25에 대한 용어집에 새로 업로드	결과 답변을 LLM을 사용해 보여줌

진행 상황

중간발표 이후 한 일



프로젝트 방향 재구성

파서 코드 재구현

청킹 코드 재구현

이번주 진행 현황

진행 상황

중간발표 이후 한 일



프로젝트 방향 재구성

기능적인 부분보다는 진행 방향을 먼저 잡는게 중요하다고 판단하여
회의를 통해 이에 시간 투자

이번주 진행 현황

진행 상황

중간발표 이후 한 일



프로젝트 방향 재구성

파서 코드 재구현

hwp, docx, pdf 파서 코드를 기존에는 청킹코드와 같이 진행했지만,
코드를 분리해, json 한줄로 파서 결과가 나오도록 코드 재구현

이번주 진행 현황

프로젝트 방향 재구성

파서 코드 재구현

청킹 코드 재구현

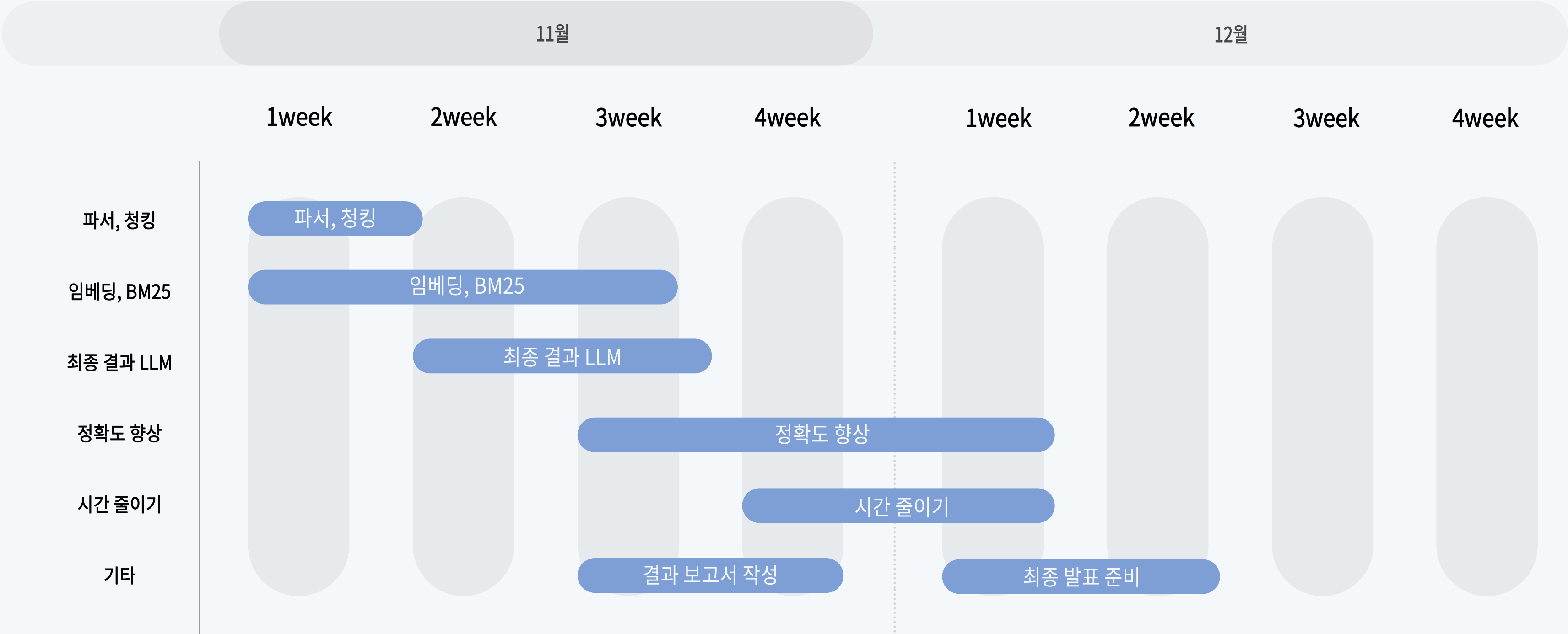
이번주 진행 현황

청킹 방식을 문장 단위 / 토큰 단위 / 오버랩 윈도우 기반 시도

->토큰 오버랩 단위로 구현시, 문장이 끊기는 경우가 많아, 문장 단위로 결정

진행 계획

최종발표 때 까지 계획





감사합니다

